

LECTURE 2

The Database Approach

CSC270 · Database Systems

COMSATS University Islamabad, Lahore Campus

In This Lecture

- The distinction between data and metadata, and why a database needs both
- The full database environment: hardware, software, data, procedures, and people
- The roles people play around a database — DBA, designers, developers, end users
- The major components a DBMS is built from
- The core functions every DBMS performs
- A first look at data dependence vs. data independence

Data and Metadata

Data is the actual facts being stored. On its own, a raw value like 4500000 tells you nothing — a database needs a second layer describing the first: **metadata** (the system catalog / data dictionary), data about the data.

Data vs. metadata, for a bank's Account table

Data

"SL21", 4500000, "B003" — the actual stored values

Metadata

accountNo is CHAR(5) and a primary key; balance is DECIMAL and must be ≥ 0 ; branchNo is a foreign key to Branch

Metadata is why a DBMS can reject bad requests before running them. When an application tries to insert a letter into balance, the DBMS looks up balance's type in the catalog, sees it's numeric, and refuses the insert — general rule enforcement, not hand-coded per column.

The Database Environment

A working database is never just “the data.” Connolly & Begg describe the environment as five interacting components:



Roles Around a Database

Role	Primary concern	Example task at a bank
Database Administrator (DBA)	Security, performance, backup/recovery, availability	Granting a new teller SELECT-only access to Account
Database Designers	Deciding what data to store and how it relates	Deciding Account needs a branchNo foreign key
Application Developers	Writing the programs end users interact with	Building the teller's “process withdrawal” screen
End Users	Using the finished application to do their job	A teller processing a customer's withdrawal

Two kinds of end user. Naive users interact only through a fixed application interface, unaware a database even exists (a teller using a withdrawal screen). Sophisticated users write their own queries directly against the database (a risk analyst querying total outstanding loans per branch).

The DBA deserves particular emphasis: in a file-based world, no single person was responsible for the organization's data as a whole — the database approach concentrates that responsibility into one role.

Components of a DBMS

Zooming in from the environment to the DBMS software itself, several cooperating components each handle a distinct concern:

Query Processor

Parses and optimizes queries submitted by users and applications

Database Manager

Coordinates access to stored data; enforces integrity and security rules

Transaction Manager

Ensures concurrent operations stay correct and recoverable

File Manager

Manages the allocation of physical disk space for stored files

DML / DDL Precompiler

Translates database statements embedded in application code

Data Dictionary / Catalog

Stores the metadata every other component consults

A single UPDATE Account SET balance = balance - 5000 WHERE accountNo = 'SL21' passes through the query processor, the catalog, the transaction manager, and finally the file manager to write the new value to disk.

Functions of a DBMS (1/2)

- **Data storage, retrieval, and update** — the most basic function: store a value, get it back, change it
- **A user-accessible catalog** — metadata is itself queryable by tools and administrators
- **Transaction support** — a group of related operations either all complete or none do
- **Concurrency control** — correctness guarantees when many users access data at once
- **Recovery services** — restoring correct state after a hardware or software failure

Functions of a DBMS (2/2)

- **Authorization services** — controlling who is allowed to do what, down to individual tables or columns
- **Support for data communication** — integrating with the network software applications use to reach the database
- **Integrity services** — enforcing that stored data satisfies the organization's business rules
- **Services to promote data independence** — insulating applications from physical storage details
- **Utility services** — import/export, performance monitoring, and analysis tools

Data Dependence vs. Data Independence — A First Look

File-based systems suffered from program-data dependence: application code was written assuming an exact physical file layout. Data independence is the DBMS's answer.

Data dependence (file-based)

Application code hard-codes field order and size; any physical change breaks every program that reads the file

Data independence (database approach)

Applications see a stable logical view; the DBMS absorbs physical changes underneath it

Don't confuse “independence” with “the schema never changes.” Data independence does not mean the database's structure is frozen. It means changes at one level can be made without cascading into forced changes at another level. Lecture 4 splits this into logical and physical data independence, once the ANSI-SPARC three-schema architecture is on the table.

Key Takeaways

- Metadata (the system catalog) describes the data and lets a DBMS enforce rules generically rather than through hand-coded logic
- The database environment is hardware, software, data, procedures, and people together — excellent table design can still fail if procedures or people are neglected
- Distinct roles — DBA, database designers, developers, and end users (naive and sophisticated) — divide responsibility for a shared database
- A DBMS is built from cooperating components (query processor, database manager, transaction manager, file manager, catalog) delivering storage, transactions, concurrency, recovery, security, integrity, and data independence
- Data independence lets one level change without forcing dependent applications to change — expanded fully in Lecture 4